

UNIVERSITÀ
DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Corso di Laurea in
Informatica

Elaborato finale

Sviluppo ed Integrazione di Knowledge Graph Multilingua

Una conoscenza condivisibile e riutilizzabile

Supervisore
Giunchiglia Fausto

Co-Supervisore
Zamboni Alessio

Laureando
Novel Alessio

Anno accademico 2023/2024

Ringraziamenti

A Fabrizio, Sabrina, Simone, Manuela, Manuela, Silvana, Galliano, Alda e Fulvio. Grazie per non avermi mai fatto mancare nulla e per avermi dato tutto il supporto di cui avevo bisogno in ogni mia decisione e difficoltà. Non potrei immaginare una famiglia migliore e sono profondamente grato di avervi accanto, sia nei ricordi che nella vita quotidiana.

Indice

Sommario **2**

1 Introduzione **2**
1.1 La crescita dell'esigenza dei Big Data 2
1.2 La metodologia iTelos 3
1.3 Struttura della tesi 3

2 I lessici multilingua **4**
2.1 Il problema 4
2.2 Struttura degli MLDB 4
2.3 Universal Knowledge Core 5
2.4 Lessici multilingua all'interno di iTelos 6

3 Requisiti KarmaLinker **7**
3.1 Requisiti generali 7
3.2 Requisiti specifici del lavoro di tirocinio 8

4 Processi KarmaLinker **9**
4.1 Creazione di un modello di replay 9
4.1.1 Mapping 11
4.2 Applicazione di un modello di replay 12
4.3 Applicazione di un modello di replay a dati del catalogo 12

5 Architettura Logica KarmaLinker **13**

6 Caso di studio e tutorial **14**
6.1 Importazione dei dati e delle ontologie 15
6.2 Creazione di un modello 17
6.2.1 Trasformazione dei dati 17
6.2.2 Modellazione dei dati 22
6.3 Applicazione di un modello 25
6.4 Impostazioni del programma 26
6.5 Documentazione API backend 28

7 Conclusioni **29**

Bibliografia **30**

Sommario

In un'epoca in cui i dati sono sempre più abbondanti e diversificati, l'integrazione dei dati provenienti da fonti eterogenee rappresenta una sfida complessa e cruciale. Questo elaborato si propone di affrontare tale sfida proponendo un approccio innovativo che combina l'uso di lessici multilingua e l'applicazione di metodologie semantiche avanzate. Il lavoro si focalizza su due aspetti principali: lo sviluppo della metodologia iTelos per l'integrazione semantica dei dati e l'implementazione del software KarmaLinker.

La metodologia iTelos si distingue per la sua capacità di superare le barriere semantiche e linguistiche, facilitando la condivisione e il riutilizzo dei dati integrati. Il processo di iTelos prevede una serie di fasi strutturate, dalla raccolta e definizione delle ontologie, fino alla modellazione e integrazione formale dei dati, garantendo una gestione ottimale dell'informazione. Questo approccio si è dimostrato particolarmente efficace nel risolvere i problemi legati alla diversità semantica, assicurando una maggiore precisione e coerenza dei dati integrati.

Il software KarmaLinker, analizzato in dettaglio e costituente la parte principale di questo elaborato, rappresenta uno strumento innovativo per l'integrazione dei dati. Grazie alla sua interfaccia intuitiva e alle funzionalità avanzate, KarmaLinker consente di assegnare significati semantici ai dati in modo semplice ed efficiente. Questo strumento ha dimostrato di migliorare significativamente l'efficacia dei processi di integrazione, riducendo i tempi di implementazione e aumentando la qualità dei risultati.

Infine, viene presentato un caso di studio che illustra in dettaglio l'applicazione concreta di KarmaLinker, evidenziando l'efficacia delle soluzioni proposte. Durante il mio tirocinio, ho contribuito allo sviluppo di diverse funzioni e alla creazione della documentazione del software, migliorando ulteriormente le capacità di integrazione dei dati. Le funzionalità e la documentazione da me implementate sono descritte e analizzate, mostrando come queste abbiano facilitato l'uso e la comprensione di KarmaLinker, oltre a dimostrarne il valore pratico in contesti reali.

1 Introduzione

1.1 La crescita dell'esigenza dei Big Data

Negli ultimi anni l'utilizzo dei Big Data (ovvero raccolte di dati caratterizzati da volume, varietà, velocità e veridicità) hanno rivoluzionato lo sviluppo e l'utilizzo delle applicazioni moderne, migliorando la capacità di prendere decisioni informate e permettendo lo sviluppo di applicazioni innovative come i Large Language Model. [2]

Assieme all'incremento nell'utilizzo dei Big Data, è cresciuto anche il bisogno nell'avere a disposizione delle tecnologie che permettano di integrare assieme dati provenienti da fonti e lingue diverse.

Più nello specifico, partendo dallo scopo dell'integrazione dei dati (ovvero la finalità per cui i dati andranno utilizzati) e dai dati stessi, l'obiettivo è quello di ottenere delle basi di conoscenza (Knowledge Graph) che contengano tutti e soli i dati rilevanti al raggiungimento dello scopo in modo da:

- Risolvere la eterogeneità semantica.
- Ottenere un risultato che massimizzi la condivisibilità.
- Ottenere un risultato che massimizzi la riusabilità.

L'eterogeneità semantica è il fenomeno che si manifesta ogni volta che c'è la necessità di integrare due o più fonti di dati, ad esempio database, che rappresentano, possibilmente in parte, lo stesso fenomeno del mondo reale. La diversità è inevitabile in natura ed è radicata nella varietà insita nei diversi mezzi di rappresentazione. [8]

Un esempio pratico del problema dell'eterogeneità semantica è rappresentato in Figura 1.1. In

Car				
Nameplate	**schema: speed**	**schema: fuelCapacity**	**schema: fuelType**	**schema: modelDate**
FP372MK	150	62	Petrol	2020-11-25

Vettura		
Targa	**Velocità**	**Tipo di corpo**
FP372MK	158	Coupé

Vehicle			
vso:VIN	**vso:feature**	**vso:modelDate**	**vso:speed**
FP372MK	Armrest	2020-11-25	155.0

Figura 1.1: Tre schemi semanticamente eterogenei contenenti una descrizione eterogenea della stessa entità.

questo esempio è facilmente possibile notare che la stessa entità reale, identificata dall'automobile targata FP372MK, viene descritta in 3 maniere diverse in 3 database diversi.

1.2 La metodologia iTelos

La metodologia iTelos punta ad ottenere il risultato migliore nell'ottica degli obiettivi precedentemente citati. [3]

Essa è composta dalle seguenti fasi:

- Scope Definition e Inception, in cui vengono definiti i requisiti funzionali e vengono raccolte le ontologie di riferimento e i dati da integrare.
- Modelling (o Informal Modelling), nella quale si costruisce lo schema più adatto al raggiungimento dello scopo dell'integrazione.
- Knowledge Alignment (o Formal Modelling), nella quale lo schema ottenuto dalla fase precedente viene allineato con le ontologie di riferimento (ovvero schemi già esistenti e popolari).
- Data Integration, nella quale il Knowledge Graph finale viene costruito integrando i dati nello schema ottenuto dalla fase precedente.

La massimizzazione della condivisibilità viene raggiunta grazie alla terza fase, la quale fa si che lo schema del Knowledge Graph finale sia allineato con degli schemi già esistenti e popolari. La massimizzazione della riusabilità viene raggiunta nella quarta fase, applicando una Popularity Driven Data Integration. [7]

Condivisibilità e riusabilità sono dei requisiti fondamentali in quanto il processo di Data Integration, essendo molto costoso, deve essere eseguito il meno possibile. Dunque ottenere come risultato dei dati pronti a futuri ulteriori processi di Data Integration, senza il bisogno di applicare tecniche di pulizia o di preprocessamento, è di estrema importanza.

1.3 Struttura della tesi

Nel capitolo 2 vengono presentati e descritti i lessici multilingua, definendone il loro scopo all'interno della metodologia iTelos. Nei successivi capitoli viene presentato e descritto KarmaLinker ovvero il software che si occupa della fase di Data Integration e sulla quale si è basato il mio lavoro di tirocinio. Nel capitolo 3 vengono presentati i requisiti di KarmaLinker. Nel capitolo 4 vengono presentati i processi relativi a KarmaLinker. Nel capitolo 5 viene descritta l'architettura logica che sta alla base

del funzionamento di KarmaLinker. Nel capitolo 6 vengono presentati vari casi di studio e un tutorial sull'utilizzo del software KarmaLinker. Nel capitolo finale 7 vengono descritte le conclusioni del lavoro svolto.

2 I lessici multilingua

2.1 Il problema

I lessici multilingua, o Multilingual Lexical DataBase (MLDB), sono delle risorse linguistiche che raccolgono e organizzano parole e frasi provenienti da più lingue, mettendo in relazione i loro significati e usi. A differenza dei classici dizionari bilingui a corrispondenza diretta, i lessici multilingua sono in grado di mettere in relazione molteplici lingue. [6]

I requisiti che un MLDB deve soddisfare sono i seguenti:

- Spazio semantico lessicale imparziale: ovvero che il MLDB deve poter rappresentare concetti lessicali specifici della lingua per una qualsiasi delle lingue che copre.
- Relazione di equivalenza interlinguistica: ovvero che il MLDB deve poter esprimere l'equivalenza concettuale per qualsiasi coppia di lingue (tra le lingue supportate).
- Relazione di iperonimia interlinguistica: ovvero che il MLDB deve poter esprimere relazioni di tipo generico-specifico per qualsiasi coppia di lingue (tra le lingue supportate).
- Relazione di intraducibilità: ovvero che il MLDB deve rappresentare le lacune lessicali come un modo per indicare esplicitamente l'intraducibilità per qualsiasi coppia di lingue (tra le lingue supportate), distinguendola dalla mera assenza di una corrispondenza che implica incompletezza lessicale della risorsa.

Questi requisiti devono essere rispettati allo scopo di permettere di avere delle risorse multilingue, ovvero delle risorse che possono essere comprese e utilizzate senza alcun vincolo imposto dal linguaggio, ma soprattutto per porre fine al fenomeno del Digital Language Divide, che consiste nella disparità nell'accesso e nella disponibilità delle risorse digitali tra le diverse lingue, favorendo quelle più diffuse e penalizzando le lingue minoritarie, con conseguenze significative per l'inclusione digitale e la diversità linguistica.

2.2 Struttura degli MLDB

Gli MLDB nella pratica non possono essere costruiti raccogliendo e unendo molteplici dizionari bilingui a corrispondenza diretta per una questione di scalabilità. Infatti, se un MLDB supporta n lingue, allora dovrebbero essere definiti i mapping per $n(n-1)$ coppie di lingue, rendendola una scelta impraticabile.

Dunque gli MLDB vengono costruiti attorno un hub rappresentativo del significato, su cui tutti i lessici sono mappati. In questo modo se un MLDB supporta n lingue devono essere definiti i mapping verso l'hub rappresentativo per n lingue. La scelta dell'hub influisce pesantemente sulle capacità del MLDB, infatti quando lo spazio semantico centrale di un MLDB è limitato a quello di una particolare lingua (come l'inglese), significa che l'intero MLDB è orientato verso quella lingua, causando problemi di rappresentazione in quanto alcuni lessici non potranno essere rappresentati con lo stesso livello di dettaglio di altri. [5]

Per capire meglio l'importanza della scelta dell'hub rappresentativo, di seguito viene illustrato un esempio di problematica che può emergere implementando l'hub linguistico basato su una singola lingua. Utilizzando un hub basato sulla lingua inglese, infatti, la traduzione della frase in lingua italiana "Mia cugina ha sposato un uomo alto" in lingua francese porta a risultati non desiderati. È ben noto che in inglese esista solamente la parola *cousin*, ovvero senza distinzione tra il concetto maschile e femminile di cugino. Per eseguire questa traduzione, dunque, la parola italiana *cugino* verrà prima convertita nella parola inglese *cousin* (con perdita di informazione) per poi essere convertita

nella parola francese *cousin*, che è maschile, invece di *cousine*, che è femminile, cambiando il significato della frase in "Mio cugino ha sposato un uomo alto". Ovviamente questo comportamento deve essere evitato, quindi è fondamentale scegliere un hub rappresentativo che non sia dipendente da una sola lingua.

Approfondendo la struttura dei lessici multilingua, tutti gli MLDB eseguono una distinzione tra le parole (sequenze di caratteri che appartengono ad una lingua) ed il loro significato. [1]

Questo viene fatto per offrire una maggiore semplicità nell'eseguire le traduzioni e per aiutare a catturare molte più informazioni sulle lingue, consentendo una gestione più efficiente e precisa delle sfumature semantiche e culturali tra diversi idiomi. La relazione tra parole e rispettivi significati può essere uno-a-molti (polisemia) o molti-a-uno (sinonimia). Più nello specifico:

- I lexemi (le unità di significato in una lingua) sono chiamati parole.
- Una parola con un significato specifico è chiamata un senso. È importante notare che una parola può avere più significati e, di conseguenza, più sensi.
- I sensi delle parole sinonime sono collegati a un singolo synset (insieme di sinonimi) che rappresenta formalmente i sensi sinonimi come collassati in un singolo nodo.

I synset sono interconnessi in un grafo attraverso relazioni gerarchiche di iperonimia e iponimia (significato più generale e più specifico) all'interno di una stessa lingua, in modo da consentire una navigazione strutturata e semantica tra i significati delle parole, facilitando la comprensione delle relazioni tra i termini e il loro uso corretto nel contesto linguistico.

2.3 Universal Knowledge Core

Per rispettare tutti i requisiti elencati precedentemente e per far sì di ottenere un MLDB che non contenga alcun bias, la risorsa deve essere costruita in modo da:

- Usare uno spazio semantico centrale che non sia legato a una specifica lingua esistente, ma che possa immagazzinare i significati delle parole provenienti da qualsiasi lingua e cultura.
- Avere la possibilità di interconnettere i significati delle parole attraverso relazioni gerarchiche di iperonimia e iponimia (significato più generale e più specifico) all'interno dello spazio semantico centrale, in modo da fornire una conoscenza interlinguistica, ovvero una conoscenza generale, non dipendente dalla lingua.
- Esplicitare i casi in cui si riscontra perdita di informazione in una traduzione (lexical gap).

Un lexical gap si riscontra all'interno di una lingua quando un determinato concetto non è lessicalizzato, ovvero quando non esiste una parola specifica per esprimerlo. [4]

Lo stato dell'arte dei lessici multilingua è rappresentato dall'UKC (Universal Knowledge Core), ed esso rispetta tutte queste caratteristiche. L'UKC ha un'architettura a due livelli, con un livello linguistico che contiene un grafo con parole, significati e synset per ciascuna lingua, ed un livello sopra-linguistico che contiene concetti interlinguistici. Ogni concetto presente in quest'ultimo livello rappresenta un significato di parole provenienti da almeno due delle lingue costituenti, in modo che il livello dei concetti consista nell'unione di tutti i significati delle parole che sono mappati ad almeno un'altra lingua. Dunque UKC è in grado di rappresentare i significati delle parole e la loro gerarchia sia a livello interlinguistico (con i concetti) che a livello linguistico-specifico (con i synset, come visto precedentemente). [6]

Questa risorsa non si limita alla sua utilità nel campo dello studio delle lingue, ma per come è costruita può essere utilizzata come base per i sistemi di ragionamento, infatti la coesistenza di synset e concetti rende questa risorsa lessicale utilizzabile sia per osservazioni che sono indipendenti dalla lingua di espressione, sia per osservazioni che invece sono dipendenti da una specifica lingua. Inoltre questa architettura a due livelli permette a gerarchie più piccole e non allineate di coesistere con il nucleo fuso dei significati interlinguistici, dovuta all'impossibilità di raggiungere una fusione perfetta di tutti i lessici per tutte le lingue del mondo, sia per lo sforzo implicato che per consentire casi irriducibili di diversità. [9]

Un esempio di mapping che viene fatto tramite questa architettura tra le parole riguardanti il concetto parentale di cugino tra Inglese e Cinese è in Figura 2.1. Qui è possibile notare come questa

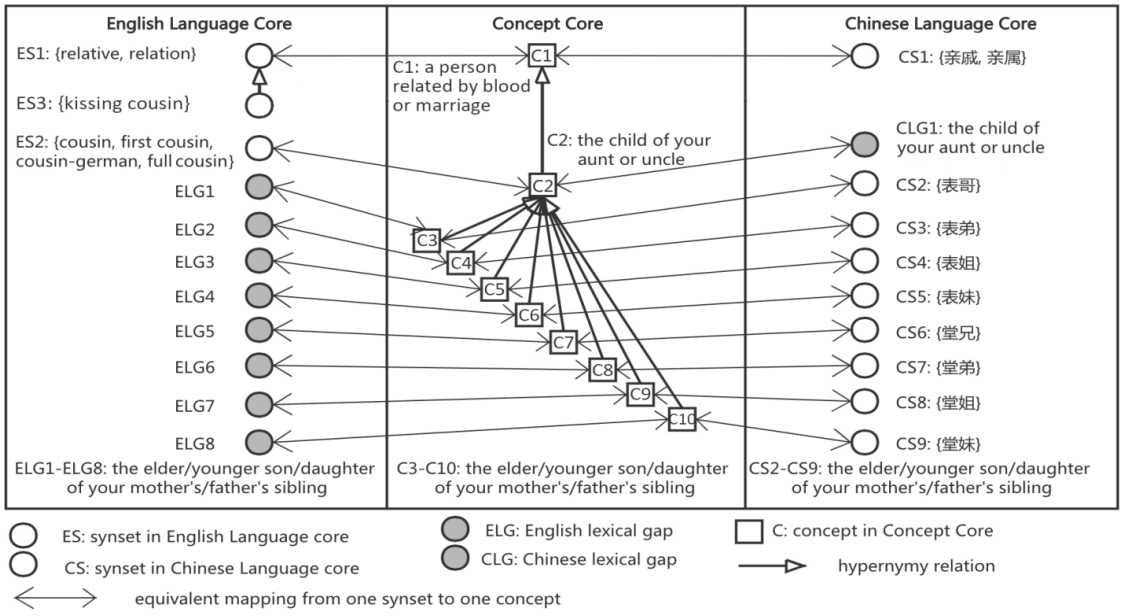


Figura 2.1: Esempio di associazioni Inglese-Cinese supportate dal UKC.

struttura permetta di esplicitare il fenomeno di lexical gap (come ELG1, ELG2, ...), di eseguire ragionamenti e trarre conclusioni che sono indipendenti dalla lingua utilizzata e soprattutto di fornire traduzioni sensate, seppur con perdita di informazione, anche nelle situazioni di intraducibilità letterale grazie alla gerarchia presente nello spazio semantico centrale.

L'architettura sottostante al UKC, infatti, permette di rappresentare:

- Equivalenza interlinguistica, mappando i synset delle specifiche lingue allo stesso concetto.

- Iperonimia e iponimia interlinguistica, grazie alla gerarchia presente nello spazio semantico centrale.

- Intraducibilità, utilizzando dei synset di gap lessicali quando un concetto non è lessicalizzato in una lingua. I synset di gap lessicali non hanno un senso o una parola associati, ma possono avere una descrizione, che permettono di descrivere un concetto anche nelle situazioni in cui non esista una parola che lo lessicalizzi direttamente. Questo permette di capire se si tratta di incompletezza della risorsa oppure di intraducibilità.

2.4 Lessici multilingua all'interno di iTelos

I lessici multilingua, in particolare UKC essendo il più sviluppato e completo, sono fondamentali nel processo di iTelos. Infatti utilizzando ontologie annotate con UKC, ovvero ontologie che sono associate ai concetti dell'UKC, è possibile ottenere come risultato finale dei Knowledge Graph multilingua, nel senso che possono essere facilmente fruibili in qualsiasi lingua grazie alla struttura di UKC. [3]

Più nello specifico, le ontologie annotate vengono date in input alla metodologia iTelos e vengono utilizzate nella fase di Knowledge Alignment per fare in modo che lo schema creato per raggiungere lo scopo dell'integrazione vada ad allinearsi con le ontologie annotate, associando ogni Entity Type con un concetto di UKC. In questa maniera, l'integrazione dei dati verrà eseguita sopra uno schema che

è una risorsa UKC, ottenendo un Knowledge Graph che gode di tutti i vantaggi che abbiamo visto precedentemente riguardo all'utilizzo dei concetti di UKC.

3 Requisiti KarmaLinker

3.1 Requisiti generali

Da questo capitolo in poi l'elaborato si concentrerà esclusivamente sull'ultimo step della metodologia iTelos, ovvero quello che esegue la Data Integration. Come già anticipato questo processo viene eseguito tramite l'uso del software KarmaLinker e il mio lavoro di tirocinio si è basato sull'aggiungere delle funzionalità ed eseguire la documentazione di questo applicativo.

Di seguito, come in ogni processo di sviluppo software, vengono elencati tutti i requisiti funzionali e non funzionali che KarmaLinker deve soddisfare, in modo da capire al meglio quali sono i servizi e l'uso che viene richiesto al prodotto.

RF1: Mapping dei dati

1. Il software deve permettere all'utente di dare in input una ontologia e un file (arbitrario) ed eseguire il mapping dei dati (nel senso che ad ogni elemento dei dati viene associato un elemento dello schema), ottenendo in output i dati mappati.

2. Il file in input può essere nei seguenti formati:

   - Formati flat: .csv
   - Formati strutturati: .json
   - Formati strutturati DAG: .xml

3. L' ontologia può essere nei seguenti formati:

   - .owl
   - .rdf
   - .xml
   - .ttl

4. Il file in output può essere nei seguenti formati:

   - .rdf
   - .json
   - .eml

RF2: Creazione di un modello KR2RML

1. Il software deve permettere all'utente, dopo aver eseguito tutte le operazioni per l'integrazione dei dati, come preprocessamento e mapping, di ottenere un modello KR2RML che permetta di replicare tutte le operazioni eseguite dall'utente sui dati.

RF3: Applicazione di un modello KR2RML

1. Il software deve permettere all'utente di dare in input, in aggiunta al file contente i dati e l'ontologia, un modello KR2RML facendo sì che il software esegua in maniera automatica tutte le operazioni per l'integrazione dei dati seguendo il modello precedentemente costruito.

2. Questo è possibile con il vincolo che i dati conferiti in input siano con la stessa struttura dei dati su cui il modello KR2RML è stato precedentemente costruito.

RF4: Supporto per lessici multilingua UKC

1. Il software deve integrare e implementare l'interfacciamento e l'integrazione con il sistema UKC all'interno dei suoi processi, in modo da rendere ogni processo e i dati generati non dipendenti da una lingua.

2. In particolare, il multilinguismo e l'integrazione con il sistema UKC sono richiesti all'interno delle strutture dati, dei dati stessi e dell'interfaccia utente.

3. Il software deve gestire le situazioni in cui in una lingua sia presente un lexical gap (la situazione in cui un concetto non è lessicalizzabile) mostrando all'utente la descrizione del concetto (se presente) al posto della lessicalizzazione mancante.

RF5: Conservazione delle informazioni semantiche

1. Il software deve mantenere e preservare le informazioni semantiche (ovvero le annotazioni) associate allo schema e ai dati durante tutto il processo di integrazione dei dati, in ogni operazione, utilizzando EML.

2. EML è uno standard XML che permette di mantenere e trasmettere le informazioni semantiche dei dati, garantendo una descrizione dettagliata e contestualizzata che ne facilita la comprensione, l'integrazione e il riutilizzo.

RF6: Utilizzo di risorse esterne

1. Il software deve consentire l'interfacciamento con SCROLL, che consente di eseguire l'estrazione dei concetti e dei compiti di NLP sui dati, abilitando l'importazione della semantica dei dati.

2. Il software deve consentire l'interfacciamento con il catalogo, il quale contiene tutte le fonti dati utilizzate per il processo.

RNF1: Tecnologie

1. Il software deve essere scritto con Java, HTML, CSS, Javascript e JQuery.

2. Il software deve essere gestito con lo strumento Maven.

3. Il software deve essere versionato e condiviso tramite la piattaforma GitLab.

RNF2: Architettura

1. Il software deve essere accessibile all'utente tramite una interfaccia web.

3.2 Requisiti specifici del lavoro di tirocinio

Di seguito vengono elencati i requisiti funzionali e non funzionali che ho dovuto soddisfare durante il mio lavoro di tirocinio su KarmaLinker.

RF7: Visualizzare le descrizioni dei concetti

1. Il software deve consentire all'utente, nel momento in cui l'utente posiziona il cursore del mouse su un concetto durante l'operazione di selezione del tipo semantico, di visualizzare la descrizione del concetto con un pop-up.

2. La descrizione del concetto deve essere recuperata tramite l'utilizzo di un'API esterna utilizzando come parametro il GID del concetto. L'API esterna viene specificata tramite il microservizio "SCROLL API".

RF8: Selezione della lingua

1. Il software deve consentire all'utente di selezionare, tra le lingue disponibili, la lingua in cui visualizzare la descrizione dei concetti (come descritto precedentemente).
2. La lista delle lingue disponibili viene fornita tramite l'utilizzo di un'API esterna. L'API esterna viene specificata tramite il microservizio "SCROLL API".

RF9: Importazione dell'ontologia di SCROLL

1. Il software deve consentire all'utente di utilizzare come ontologia quella messa a disposizione dall'API esterna specificata dal microservizio "SCROLL API".

RF10: Documentazione delle API

1. Il software e lo scopo riguardante le API del backend, che consistono nelle uniche modalità con cui è possibile interagire con il backend di KarmaLinker, deve essere documentato in maniera esaustiva.
2. Per la documentazione bisogna utilizzare lo standard OpenAPI.

RNF3: Struttura del codice

1. Per effettuare le chiamate alle API esterne bisogna utilizzare (o creare) una Servlet nel backend dedicata che funzioni da proxy. Dunque il frontend potrà scambiare informazioni solamente con la componente di backend di KarmaLinker, la quale si occuperà di interagire con tutti i servizi esterni necessari.
2. Per tutte le funzioni del backend, utilizzare le tecnologie e la logica del software originale.
3. Per tutte le funzioni del frontend, mantenere l'aspetto grafico e la logica del software originale.
4. Seguire i Java Design Patterns per il codice scritto in Java. [14]

4 Processi KarmaLinker

In questo capitolo vengono analizzati i 3 processi principali che il software KarmaLinker permette di eseguire, elencando per ognuno di essi lo scopo, la sequenza di attività di cui è composto ed il risultato finale. I 3 processi in questione sono:

- Creazione di un modello di replay.
- Applicazione di un modello di replay su dati esterni.
- Applicazione di un modello di replay su dati interni.

4.1 Creazione di un modello di replay

Il primo processo consiste nel creare un modello KR2RML, detto anche Replay Description Model. KR2RML è un linguaggio usato per descrivere il mapping semantico, che supporta sia dati tabulari, come CSV, che gerarchici, come XML e JSON, convertendoli in un modello comune chiamato Nested Relational Model. Questo linguaggio facilita le operazioni di pulizia, trasformazione e mapping dei dati, permettendo l'utilizzo di trasformazioni complesse come quelle di NLP (Natural Language Processing). [12]

Per creare il Replay Description Model, l'utente, dopo aver scelto i dati e l'ontologia su cui lavorare, eseguirà tutte le operazioni necessarie sui dati per poi, alla fine, eseguire il mapping tra i dati e

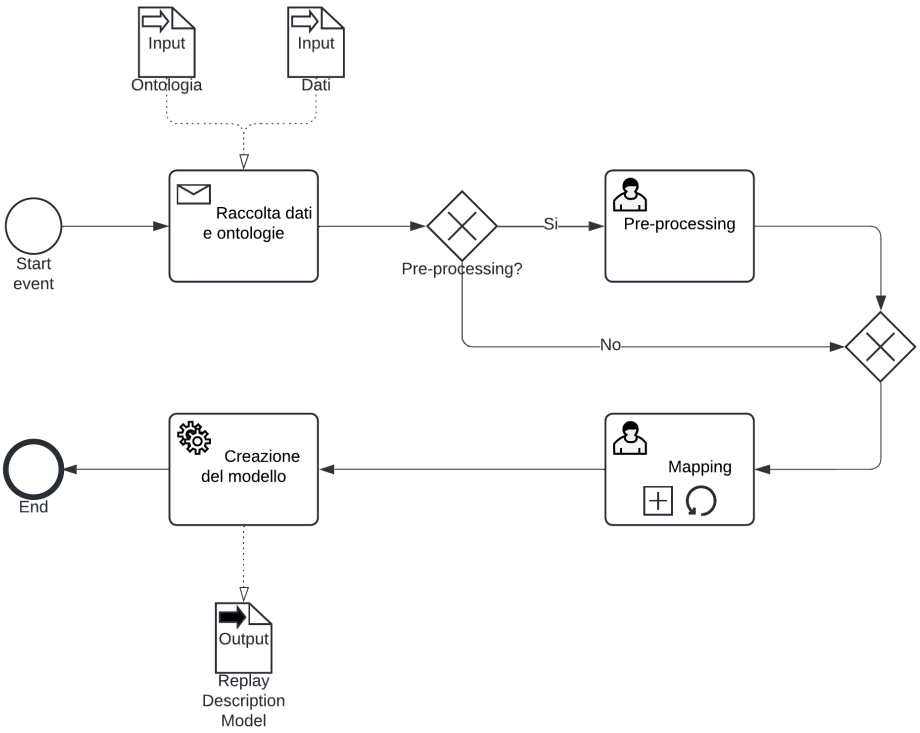


Figura 4.1: Rappresentazione del processo di creazione di un Replay Description Model.

l'ontologia. Il modello costruito sarà in grado di replicare tutte le operazioni eseguite dall'utente in maniera automatica, in modo da permettere l'integrazione dei dati su dati futuri senza la necessità dell'intervento dell'utente. Il processo è rappresentato in Figura 4.1.

Più nello specifico, in questo processo l'utente, come prima cosa, deve fornire in input a KarmaLinker le ontologie e i dati con cui intende eseguire l'attività di Data Integration. I dati in input possono essere nei seguenti formati:

- .csv
- .json
- .xml

Una volta che i dati da integrare sono stati caricati su KarmaLinker, l'utente può decidere se eseguire o meno delle attività di preprocessing sui dati. Le azioni di preprocessing consentite sono le seguenti:

- Add Column, permette di aggiungere una nuova colonna con un valore predefinito.
- Rename, permette di rinominare la colonna.
- Split Values, permette di creare una nuova colonna o aggiornare una colonna con valori che sono il risultato della divisione dei dati in base a un delimitatore.
- Add Row, aggiunge una nuova riga vuota alla tabella.
- PyTransform, permette di scrivere espressioni in Python per definire trasformazioni dei dati.
- Aggregation, permette di aggregare i dati basandosi su una funzione predefinita o definita dall'utente.
- Unfold, permette di trasporre le righe in colonne.
- Fold, è l'opposto di Unfold e permette di trasporre le colonne in righe.

- Glue Columns, permette di combinare più tabelle annidate in una singola tabella.
- Group By, permette di raggruppare i dati per le colonne selezionate.
- Selection, permette di selezionare solo un sottoinsieme di dati eseguendo una funzione Python su di esso.

Dopo le operazioni opzionali di preprocessing, l'utente esegue effettivamente l'operazione di Data Integration, effettuando il mapping dei dati sulle ontologie. In questa operazione l'utente è assistito dal software che, per ogni colonna dei dati, presenta all'utente la lista di concetti (recuperata dall' ontologia) tra cui scegliere per eseguire il mapping della colonna su una proprietà di un concetto.

Una volta terminata l'operazione di mapping, viene creato il Replay Description Model, il quale contiene le informazioni per replicare tutte le operazioni di preprocessing e di mapping che sono state eseguite dall'utente.

4.1.1 Mapping

L'attività di mapping è di fondamentale importanza in quanto è l'attività che permette a KarmaLinker di raggiungere il suo scopo principale: l'integrazione dei dati. Per questo motivo l'attività viene esplicitata e approfondita in Figura 4.2.

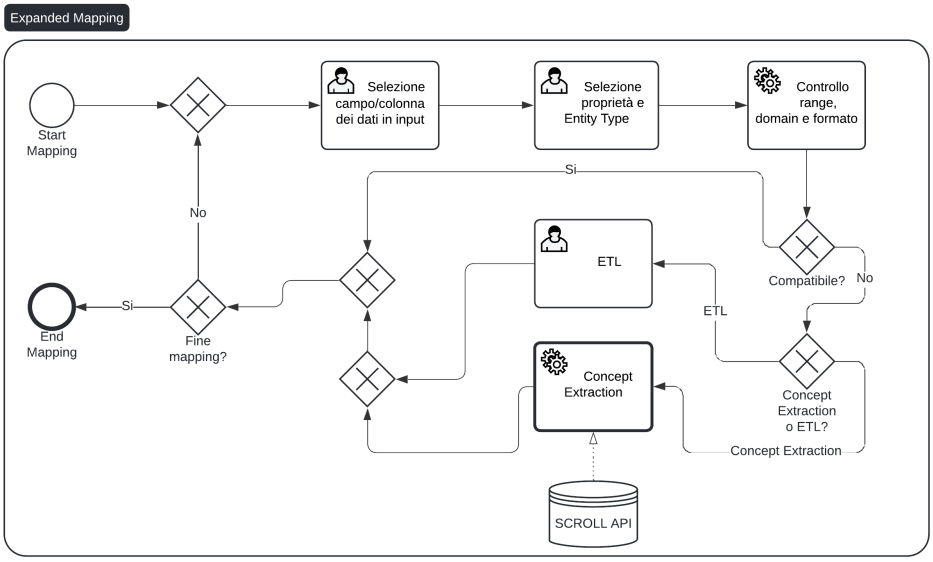


Figura 4.2: Espansione della attività di mapping.

All'interno della attività di mapping, l'utente, dunque, eseguirà tutte le operazioni necessarie per eseguire la mappatura, prendendo in considerazione una colonna/campo dei dati in input alla volta. Per colonna/campo si intende una singola unità di dati all'interno della struttura dei dati in input, che può rappresentare un singolo attributo o una proprietà specifica. Una volta selezionata la colonna/campo, l'utente dovrà indicare la proprietà e l'entity type a cui questa proprietà corrisponde, selezionandoli dall'ontologia fornita in input. Successivamente, il sistema eseguirà un controllo per verificare che ci sia compatibilità tra la colonna e la proprietà selezionata in termini di range, dominio e formato. Se questa compatibilità non è presente l'utente potrà scegliere se tentare di risolvere il problema tramite operazioni di ETL (Extract Transform Load) oppure eseguendo un'operazione di Concept Extraction.

Tutto il processo riguardante le colonne/campi che è appena stato illustrato viene ripetuto fino a che non è stato eseguito il mapping per tutte le colonne/campi del file in input. Tutte le azioni compiute dall'utente in questa fase saranno salvate in modo da poter poi, alla fine del mapping, costruire un modello che sia in grado di replicarle tutte.

4.2 Applicazione di un modello di replay

Il secondo processo consiste nell'applicare un modello KR2RML, precedentemente creato, eseguendo effettivamente l'operazione di integrazione dei dati.

Per eseguire l'integrazione dei dati, l'utente dovrà specificare quale modello applicare. Dopo aver selezionato i dati su cui applicare il modello, il programma eseguirà tutte le operazioni previste dal modello stesso, effettuando il mapping tra i dati e l'ontologia. Il processo è rappresentato in Figura 4.3.

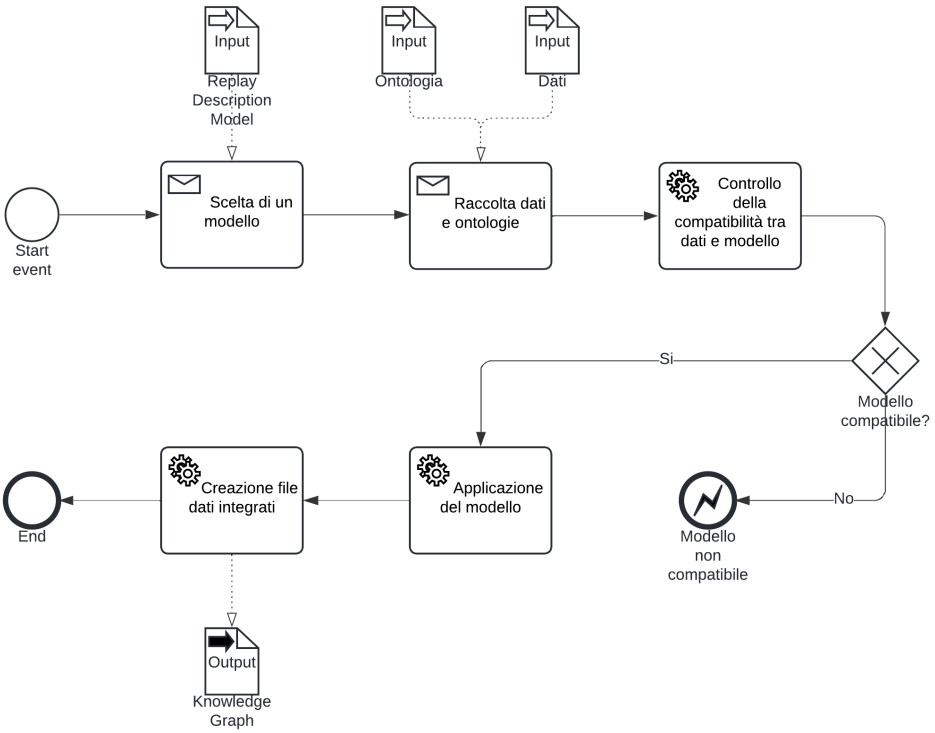


Figura 4.3: Rappresentazione del processo di applicazione di un Replay Description Model.

Più nello specifico, in questo processo l'utente, come prima cosa, deve fornire in input a KarmaLinker il Replay Description Model, che deve essere costruito precedentemente su dati aventi la stessa struttura dei dati che verranno forniti in input. Successivamente, l'utente specificherà su quali dati e ontologia eseguire l'integrazione dettata dal modello, e il software eseguirà un controllo per garantire che i dati in input siano effettivamente compatibili con il modello selezionato. In caso di esito positivo, il programma procederà con l'applicazione del modello ai dati di input, replicando tutte le operazioni contenute nel modello e ottenendo come output finale i dati mappati ed integrati.

4.3 Applicazione di un modello di replay a dati del catalogo

Il terzo processo consiste nell'applicare un modello KR2RML, precedentemente creato, su dei dati già presenti all'interno del sistema, ovvero già mappati ed integrati.

L'esplicitazione e la differenziazione che viene fatta per questo processo, rispetto al processo precedente, è dovuta al fatto che, a livello pratico, il trattamento di dati grezzi o non precedentemente elaborati, rispetto al trattamento di dati già elaborati ed integrati, risulta enormemente differente.

Il processo è rappresentato in Figura 4.4.

Più nello specifico, in questo caso i dati in input non sono in un formato "standard", ma sono recuperati dal catalogo di KarmaLinker. Dunque, in questa situazione, il modo in cui i dati vengono importati nel sistema differisce notevolmente: non è l'utente a importare i dati da integrare, ma i

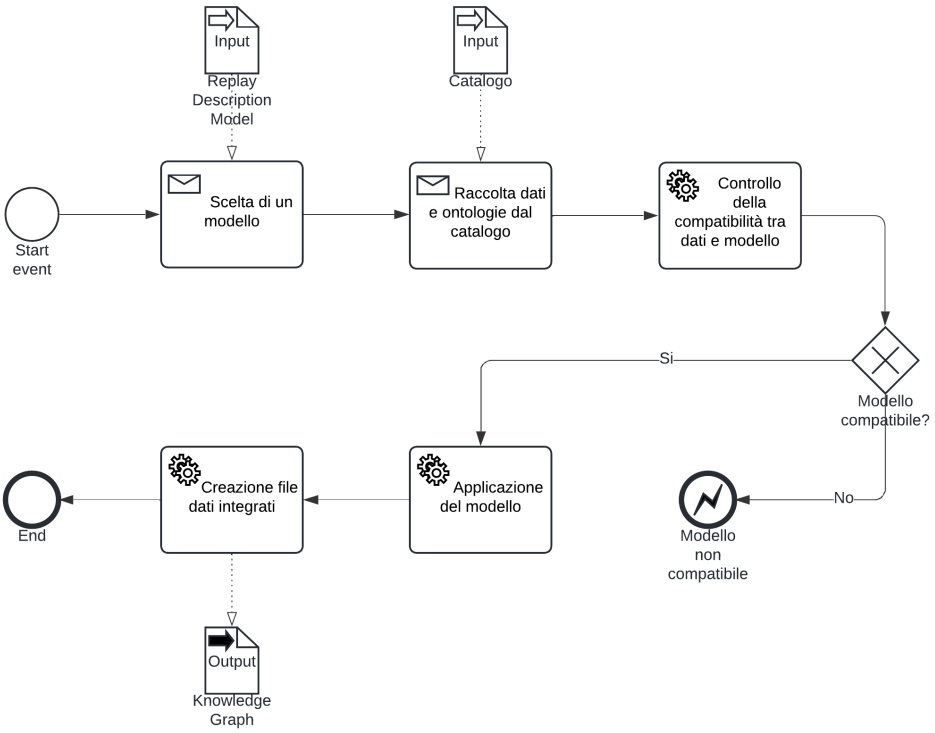


Figura 4.4: Rappresentazione del processo di applicazione di un Replay Description Model su dati interni.

dati vengono recuperati direttamente da KarmaLinker, creando e rendendo così possibile un loop di integrazione.

5 Architettura Logica KarmaLinker

L'architettura logica di KarmaLinker è raffigurata in Figura 5.1. L'architettura logica del software è caratterizzata da una notevole semplicità, principalmente perché il sistema non richiede un alto numero di interazioni con componenti esterni.

Nell'architettura logica, KarmaLinker è stato suddiviso nelle sue due componenti principali, ovvero il frontend e il backend, poiché queste possono essere considerate indipendenti l'una dall'altra.

- Frontend: Questa parte del software si occupa dell'interazione con l'utente. Tramite il frontend, l'utente può interagire con le varie funzionalità del software utilizzando un'interfaccia grafica. Il frontend consente all'utente di caricare i dati nel software e di scaricare i dati mappati dopo aver eseguito l'integrazione. Il frontend comunica continuamente con il backend per eseguire le operazioni richieste dall'utente attraverso l'interfaccia grafica.

- Backend: Questa parte del software si occupa di implementare tutte le funzioni messe a disposizione da KarmaLinker. Nel backend vengono eseguite tutte le operazioni richieste dall'utente tramite il frontend. Il backend comprende l'intera logica dell'applicazione e tutte le librerie necessarie per eseguire le varie operazioni.

I dati in input sono caricati dall'utente attraverso il frontend, il quale si occupa di inoltrarli al backend per salvarli nelle aree di memorie dedicate all'applicazione. Allo stesso modo, quando l'utente ha terminato di eseguire tutte le operazioni e richiede il file contenente i dati integrati, questa richiesta viene gestita dal backend e il file è fornito all'utente tramite il frontend.

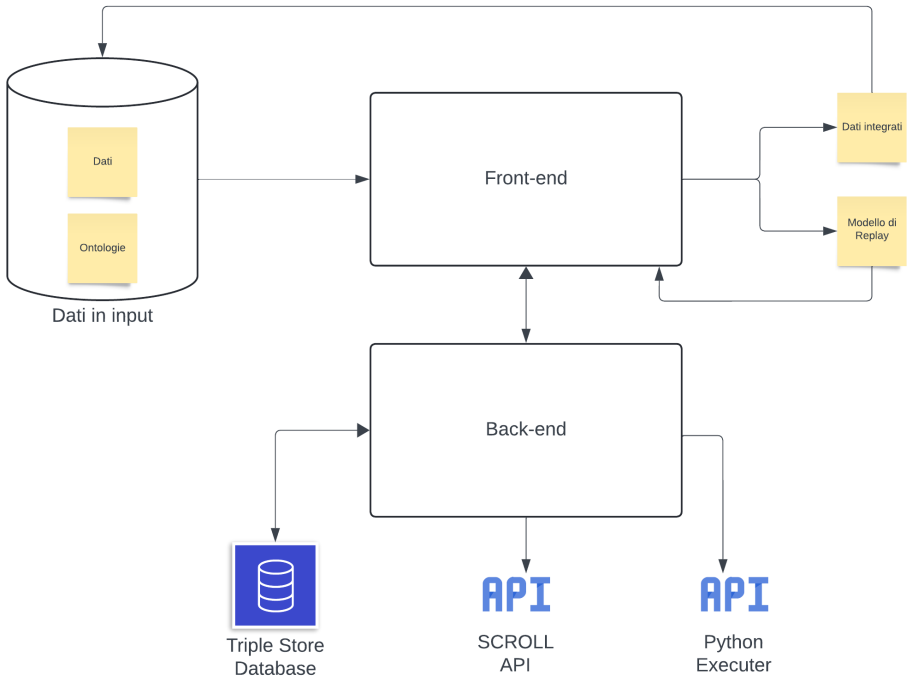


Figura 5.1: Architettura logica di KarmaLinker.

Il backend si occupa di interfacciarsi con tre principali servizi esterni:

- Triple Store Database: È il componente che gestisce i dati strutturati come triplestore RDF. Esso serve per memorizzare informazioni in una forma interconnessa, permettendo query e analisi complesse su dati relazionati ed è ideale per salvare la rappresentazione interna dei dati, come modelli di conoscenza, ontologie o altre informazioni strutturate. Questa struttura è usata per rappresentare relazioni tra entità in modo flessibile e interconnesso.

- SCROLL API: È il componente che permette di interagire con SCROLL. SCROLL (Semantic CROss-Lingual Label parser) è un servizio specializzato di elaborazione del linguaggio naturale (NLP) sviluppato per gestire la disambiguazione del senso delle parole e per l'analisi semantica di dati strutturati e multilingue. SCROLL viene integrato all'interno di KarmaLinker con l'obiettivo di eseguire il mapping tra i dati nei linguaggi naturali e l'UKC, arricchendo i dati con la componente semantica.

- Python executer: È il componente che esegue gli script in codice Python utilizzati per eseguire delle elaborazioni avanzate sui dati.

6 Caso di studio e tutorial

In questo capitolo verrà analizzato e spiegato a livello pratico l'utilizzo del software KarmaLinker, prendendo come caso di studio un dataset e un'ontologia riguardante il campo sanitario. Più nello specifico verrà spiegato come personalizzare le varie impostazioni del software e come eseguire i vari task. Verranno poi illustrati degli esempi pratici in modo da permettere una migliore comprensione delle funzionalità offerte dall'applicativo.

6.1 Importazione dei dati e delle ontologie

Per poter iniziare a eseguire delle operazioni sui dati, è necessario, come prima cosa, importare all'interno dell'ambiente di KarmaLinker i dati e le ontologie che si vogliono utilizzare per l'integrazione dei dati. KarmaLinker offre la possibilità di utilizzare diverse modalità di importazione per quanto riguarda le ontologie e i dati. Nella Figura 6.1 sono rappresentate tutte le opzioni che KarmaLinker

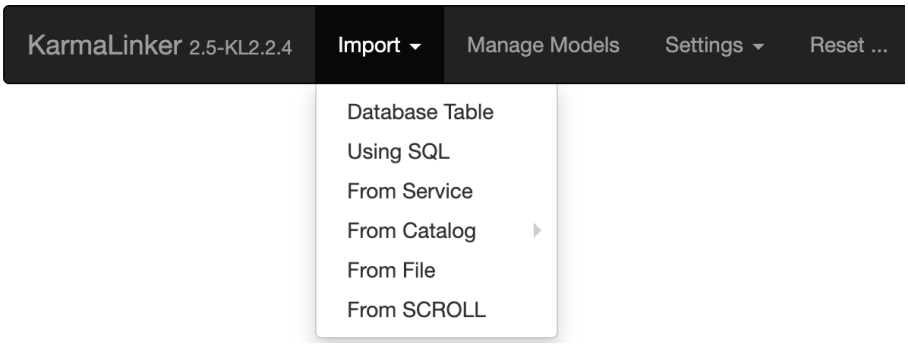


Figura 6.1: Menù di import.

mette a disposizione all'utente per effettuare l'importazione dei dati e delle ontologie.

Tramite la voce *Database Table*, KarmaLinker consente all'utente di importare i dati direttamente da una tabella appartenente ad un database relazionale. In particolare, è possibile importare dati dai seguenti sistemi di gestione di database: MySQL, SQL Server, Oracle, PostGIS, Sybase e Ingres. La schermata di importazione dei dati da una tabella appartenente ad un database è raffigurata in Figura 6.2. L'utente, dunque, dopo aver specificato il tipo di database, l'indirizzo, la porta, lo



Figura 6.2: Schermata di import da tabella di database.

username, la password e il nome del database, potrà visualizzare a schermo tutte le tabelle che sono presenti all'interno del database specificato. Da questa schermata l'utente potrà scegliere quali tabelle del database importare all'interno di KarmaLinker.

Tramite la voce *Using SQL*, KarmaLinker consente all'utente di importare i dati da una tabella appartenente ad un database relazionale, applicando però un filtro dettato da una query SQL scelta dall'utente. Il funzionamento è simile all'opzione *Database Table* vista precedentemente, con la differenza che in questo caso l'utente può filtrare i dati da importare tramite l'uso di una query SQL a scelta, come raffigurato in Figura 6.3.

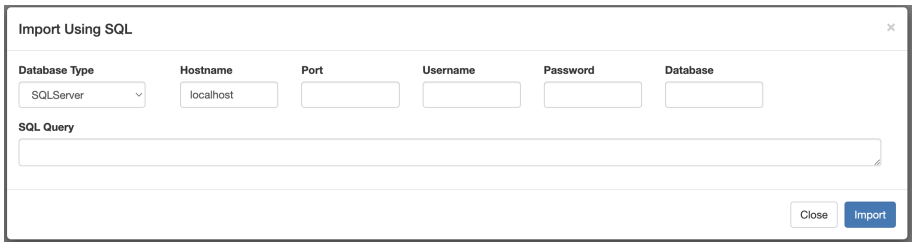


Figura 6.3: Schermata di import da database tramite query SQL.

Tramite la voce *From Service*, KarmaLinker permette all'utente di importare i dati tramite una chiamata a un servizio API web. In questo modo, KarmaLinker invierà una richiesta a un'API esterna e importerà i dati ricevuti come risposta direttamente nell'ambiente di lavoro.

Tramite la voce *From Catalog*, KarmaLinker consente all'utente di importare i dati direttamente dal catalogo interno. Ciò permette di rielaborare dati precedentemente integrati, creando un ciclo di integrazione completamente gestibile all'interno dello stesso ambiente.

Tramite la voce *From File*, KarmaLinker consente all'utente di importare i dati tramite il caricamento di un file. Una volta caricato il file, sarà visualizzato a schermo un menù nel quale l'utente dovrà selezionare il formato del file caricato, come raffigurato in Figura 6.4. Per ogni tipologia di file

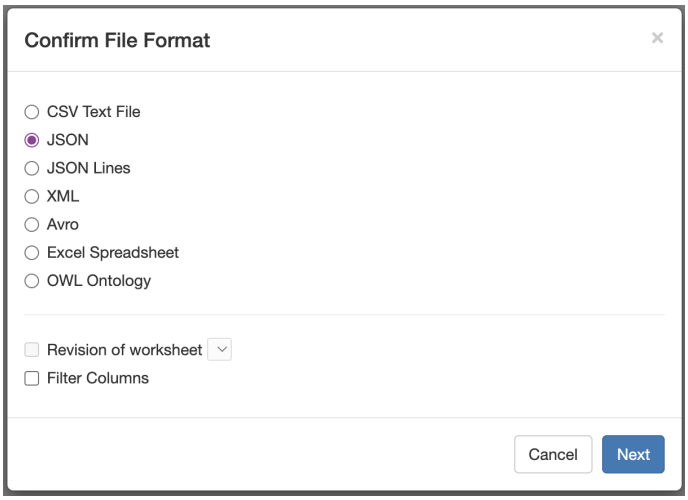


Figura 6.4: Schermata di import da file.

verranno visualizzate diverse opzioni per consentire un maggiore controllo sull'importazione, come ad esempio la scelta del carattere delimitatore, le righe che contengono gli header delle colonne e le righe che contengono i record.

Nel caso di un file di dati, l'utente potrà selezionare il numero di righe da importare nell'ambiente. Questa opzione è utile perché consente di eseguire la trasformazione e la modellazione dei dati visualizzando inizialmente solo un dataset di esempio, per poi applicare successivamente il modello creato all'intero dataset.

L'ultima voce, ovvero *From SCROLL*, è stata implementata da me durante il lavoro di tirocinio. Essa semplifica il processo di ricerca e annotazione di un'ontologia, permettendo di utilizzare direttamente un'ontologia aggiornata fornita dal servizio SCROLL. Per utilizzare questa funzione, è necessario che il microservizio SCROLL API (che verrà descritto più avanti nell'elaborato) sia configurato con un URL valido.

Una volta importati i dati utilizzando una qualsiasi delle modalità appena elencate, l'utente visualizzerà la schermata illustrata in Figura 6.5. Da qui l'utente ha la possibilità di navigare all'interno

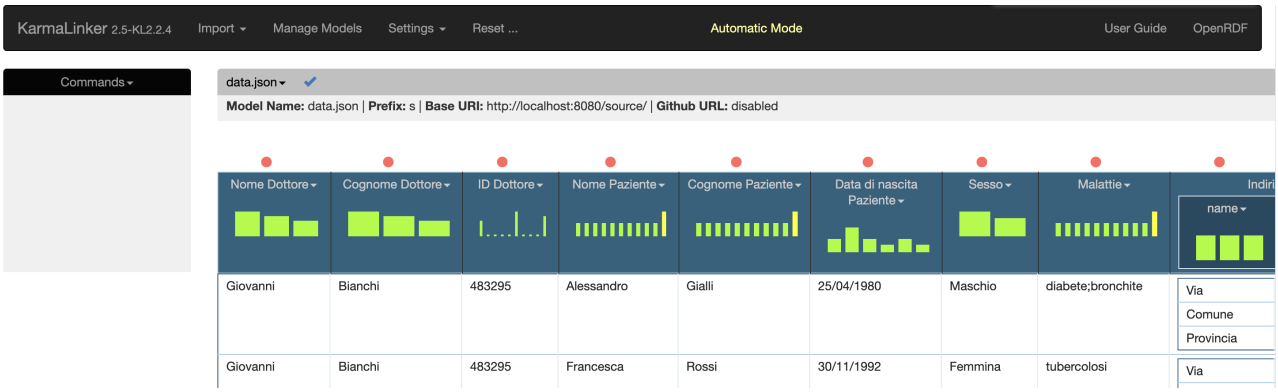


Figura 6.5: Schermata successiva all'import dei dati.

dei dati e di modificare i valori del dataset cliccandoci sopra.

Per ogni importazione di dati eseguita, verrà creato un cosiddetto *worksheet*. Un worksheet è quindi associato a un file di importazione e include il dataset, l'elenco dei comandi eseguiti e tutti i file generati a partire da quel dataset. Il worksheet verrà nominato con lo stesso nome del file di input associato.

È possibile eliminare un worksheet dall'ambiente di lavoro cliccando sopra al nome del worksheet e selezionando l'opzione *Delete Worksheet*, come illustrato in Figura 6.6.

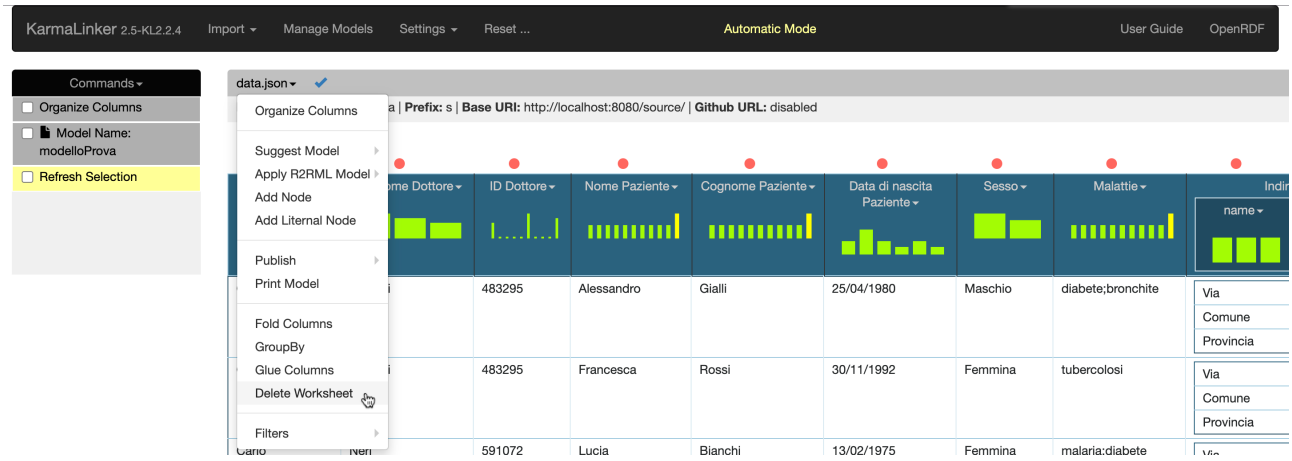


Figura 6.6: Selezione dell'opzione Delete Worksheet.

All'interno del worksheet è possibile, se necessario, modificare il Base URI, che funge da radice per tutti gli URI che verranno generati da KarmaLinker per quel worksheet. È anche possibile modificare il prefisso, ovvero la stringa utilizzata per abbreviare gli URI e rendere più facile la loro gestione e lettura.

Di default, il nome del modello finale corrisponde al nome del worksheet. Tuttavia, l'utente ha la possibilità di modificarlo a proprio piacimento. È consigliabile rinominare il modello per rendere più chiaro il suo scopo.

6.2 Creazione di un modello

Una volta importati i dati necessari in un worksheet, l'utente può iniziare a eseguire varie funzioni che contribuiranno alla costruzione del modello finale. Il modello viene creato registrando tutte le operazioni, sia di trasformazione sia di modellazione, che l'utente applica ai dati all'interno del worksheet.

6.2.1 Trasformazione dei dati

In questa sezione verranno elencate e spiegate tutte le trasformazioni che KarmaLinker permette di applicare ai dati.

Organizzazione delle colonne: La struttura del file di input potrebbe non corrispondere a quella desiderata dall'utente. KarmaLinker offre una funzione che consente di riordinare le colonne secondo le proprie preferenze. Per farlo, è necessario cliccare sul nome del worksheet e selezionare l'opzione *Organize Columns*. Successivamente, verrà visualizzata una lista di tutte le colonne presenti nel worksheet. L'utente potrà selezionare e trascinare le colonne con il mouse per modificare l'ordine di visualizzazione, come illustrato in Figura 6.7.

Occultazione delle colonne: Può accadere che alcune colonne nel file di input non siano necessarie per l'operazione di integrazione dei dati. KarmaLinker offre una funzione che consente di nascondere queste colonne nel worksheet a piacimento, permettendo all'utente di visualizzare solo le informazioni ritenute rilevanti. Per farlo, è sufficiente cliccare sul nome del worksheet e selezionare l'opzione *Hide Columns*. Successivamente, verrà mostrata una lista di tutte le colonne presenti nel worksheet. L'utente può cliccare sull'icona dell'occhio accanto al nome di una colonna per nasconderla, come illustrato in Figura 6.8.

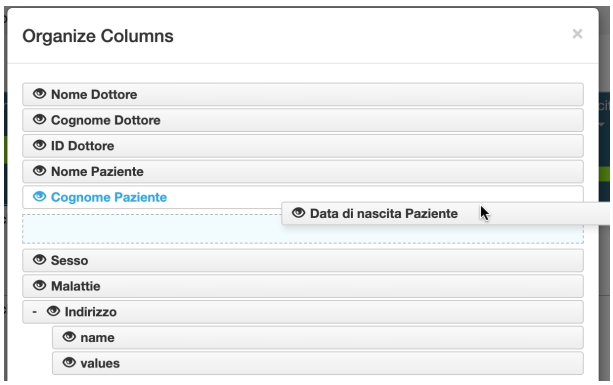


Figura 6.7: Esempio di cambiamento dell'ordine delle colonne.

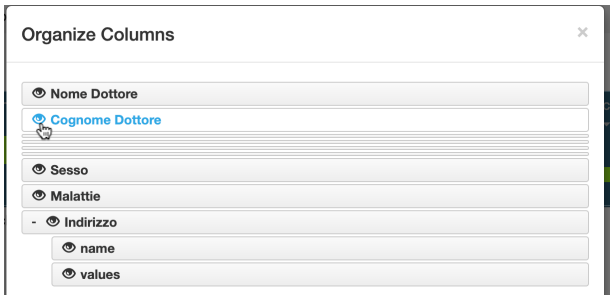


Figura 6.8: Esempio di occultazione delle colonne.

Aggiunta delle colonne: Può capitare che l'utente abbia necessità di aggiungere nuove colonne al worksheet per inserire dati mancanti. KarmaLinker offre una funzione che consente di aggiungere una colonna con un valore predefinito per ogni riga. Per farlo, l'utente deve cliccare sul nome di una qualsiasi colonna già presente e selezionare l'opzione *Add Column*, come illustrato in Figura 6.9. Successivamente, comparirà un menù a schermo che permetterà di inserire il nome della nuova colonna

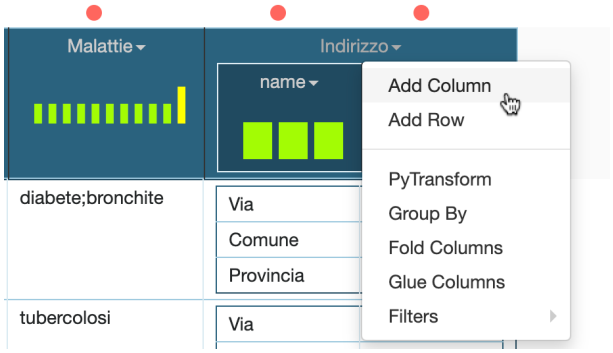


Figura 6.9: Selezione dell'opzione Add Column.

e il valore predefinito da assegnare a quell'attributo per ogni riga, come illustrato in Figura 6.10. Successivamente, sarà possibile modificare l'attributo per ogni riga cliccando sul valore che si desidera cambiare.

Aggiunta delle righe: Può capitare che l'utente abbia la necessità di aggiungere nuove righe al worksheet per inserire dati mancanti. KarmaLinker offre una funzione che consente di inserire una riga vuota, senza valori. Per farlo, l'utente deve cliccare sul nome di una qualsiasi colonna già presente e selezionare l'opzione *Add Row*, come illustrato Figura 6.11. Successivamente, sarà possibile modificare l'attributo per ogni colonna cliccando sul valore che si desidera cambiare.

Rinominazione delle colonne: Può capitare che l'utente abbia la necessità di cambiare il nome di una colonna del worksheet per renderlo più esplicativo. KarmaLinker offre una funzione che consente

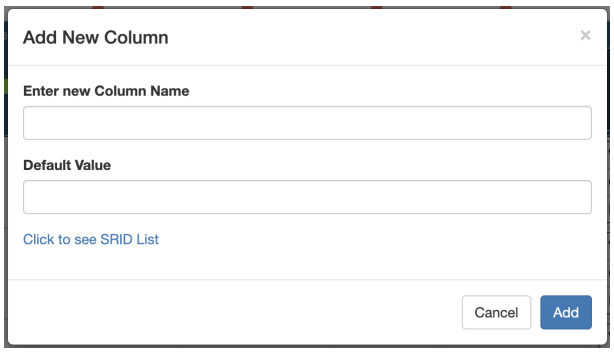


Figura 6.10: Esempio di aggiunta di una colonna.

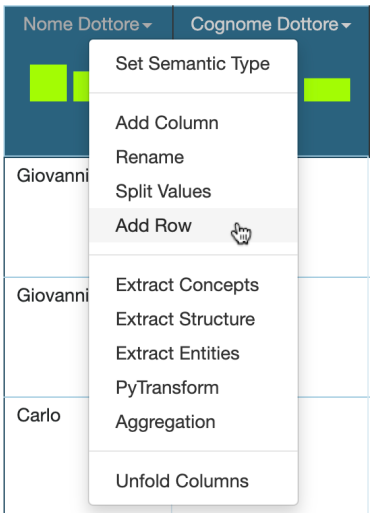


Figura 6.11: Selezione dell'opzione Add Row.

di rinominare una colonna a piacimento. Per farlo, l'utente deve cliccare sul nome di una qualsiasi colonna già presente e selezionare l'opzione *Rename* per procedere con la modifica del nome.

Dividi valori: Può capitare che l'utente abbia la necessità di dividere il valore di una colonna, che contiene più informazioni, in un vettore di valori distinti. KarmaLinker offre una funzione che permette di effettuare questa operazione. Per utilizzarla, l'utente deve cliccare sul nome della colonna su cui desidera applicare la divisione e selezionare l'opzione *Split Values*. Successivamente, apparirà un menu che permetterà all'utente di inserire il nome della nuova colonna e il carattere delimitatore, cioè il carattere tramite il quale i vari valori verranno separati, come illustrato in Figura 6.12. Il risultato,

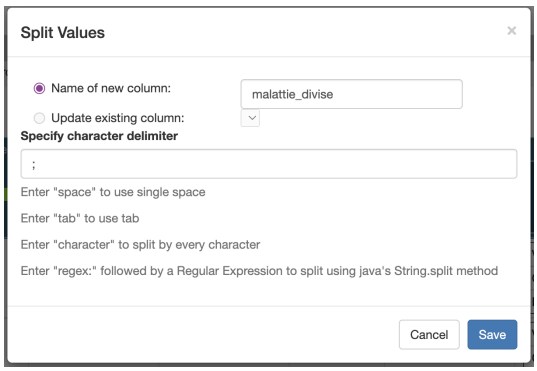


Figura 6.12: Schermata di Split Values.

che permette di trattare singolarmente i vari valori, è illustrato in Figura 6.13.

Aggregazione: Può capitare che l'utente abbia la necessità di eseguire operazioni di aggregazione

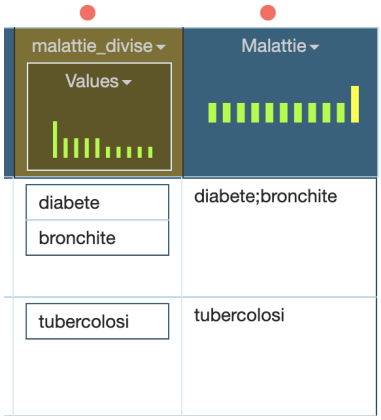


Figura 6.13: Esempio di risultato di Split Values.

come media, concatenazione, minimo o massimo. KarmaLinker offre una funzione che permette di eseguire queste operazioni di aggregazione su un array di una colonna. Per farlo, l'utente deve cliccare sul nome della colonna su cui desidera applicare l'operazione di aggregazione e selezionare l'opzione *Aggregation*. Successivamente, sullo schermo verrà visualizzato un menu che permetterà all'utente di inserire il nome della nuova colonna, scegliere la funzione di aggregazione da utilizzare e definire anche delle funzioni personalizzate per l'aggregazione, come illustrato in Figura 6.14.

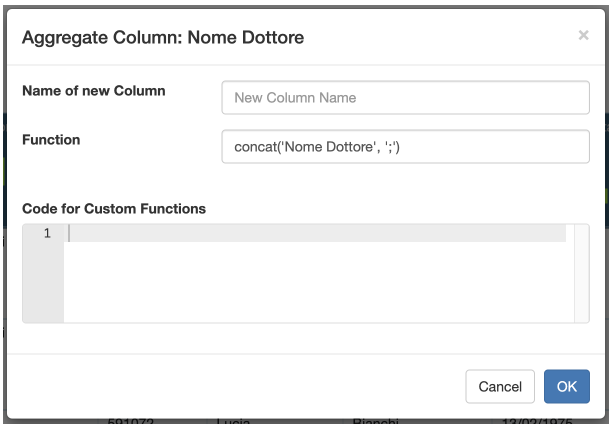


Figura 6.14: Schermata di Aggregation.

PyTransform: Può capitare che l'utente abbia la necessità di eseguire operazioni complesse sui valori del worksheet. KarmaLinker offre una funzione che consente di eseguire codice Python sui valori, permettendo così di eseguire operazioni molto complesse sui dati. Per utilizzarla, l'utente deve cliccare sul nome della colonna su cui desidera applicare l'operazione e selezionare l'opzione *PyTransform*. Successivamente, l'utente potrà inserire la funzione Python da applicare. Potrà anche decidere se modificare la colonna esistente con il risultato della funzione o crearne una nuova e visualizzare il risultato dello script applicato alle prime 5 righe per verificare se produce il risultato desiderato, come illustrato nella Figura 6.15. Un esempio del risultato dell'applicazione di PyTransform è mostrato in Figura 6.16.

Estrazione dei concetti: Può capitare che l'utente abbia la necessità di estrapolare il significato delle parole presenti in una colonna per arricchirle con la componente semantica. KarmaLinker offre una funzione che consente di eseguire l'estrazione dei concetti dai valori di una colonna. Per farlo, l'utente deve cliccare sul nome della colonna su cui desidera applicare l'operazione di estrazione dei concetti e selezionare l'opzione *Extract Concepts*. Successivamente, l'utente potrà decidere se forzare l'accoppiamento con un concetto oppure lasciarlo facoltativo, la lingua con cui estrarre il concetto e altre preferenze, come illustrato in Figura 6.17. Una volta eseguita l'operazione di Concept Extraction, sarà possibile scegliere a quale concetto specifico associare il dato, selezionandolo dalla lista dei concetti proposti, come illustrato in Figura 6.18.

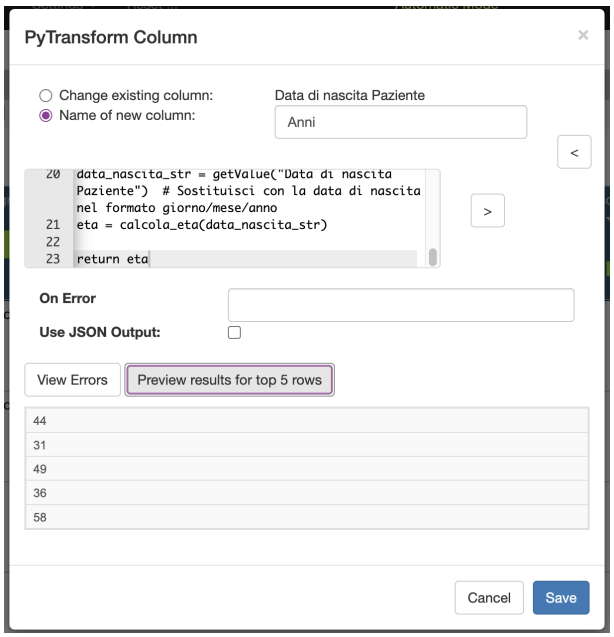


Figura 6.15: Schermata di PyTransform.



Figura 6.16: Esempio di risultato di PyTransform.

Unfold e Fold: Il dataset di input potrebbe essere strutturato in modo che i nomi delle colonne appaiano come valori di riga, o viceversa. KarmaLinker offre due funzioni che permettono di trasporre le righe sulle colonne (Unfold) e trasporre le colonne sulle righe (Fold). Per eseguirle, l'utente deve cliccare sopra alla colonna su cui vuole eseguire l'operazione di unfold/fold e selezionare le opzioni *Unfold Columns*/*Fold Columns*. Un esempio del risultato che si ottiene eseguendo l'operazione di Unfold è in Figura 6.19.

Glue Columns: Il dataset di input potrebbe essere strutturato in modo che i valori delle colonne siano dei vettori. KarmaLinker mette a disposizione una funzione che permette di trattare in diverse maniere la presenza di vettori, o eseguendo il cross product o prendendo come dimensione quella del vettore più piccolo o prendendo come dimensione quella del vettore più grande. Per farlo, è necessario cliccare sopra al nome del worksheet e selezionare l'opzione *Glue Columns*, specificando poi la modalità di trattamento e su quali colonne si intende applicare l'operazione.

Group By: L'utente potrebbe voler raggruppare i dati in base al valore di una colonna per consentire una visualizzazione più significativa del worksheet. KarmaLinker offre una funzione che permette di eseguire l'operazione di raggruppamento. Per farlo, è necessario cliccare sopra al nome del worksheet e selezionare l'opzione *Group By*, specificando su quali colonne si intende basare l'ope-

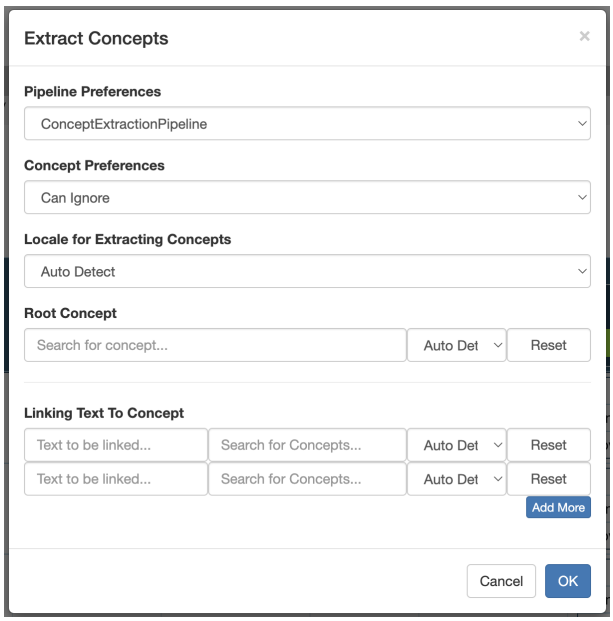


Figura 6.17: Schermata di Extract Concepts.



Figura 6.18: Esempio di risultato di Extract Concepts.

razione di raggruppamento. Un esempio di risultato dell'applicazione dell'operazione di Group By è in Figura 6.20.

6.2.2 Modellazione dei dati

In questa sezione verranno elencate e spiegate tutte le modellazioni che KarmaLinker permette eseguire sui dati.

Dunque, una volta eseguite tutte le trasformazioni necessarie sui dati presenti nel worksheet, è possibile per l'utente cominciare con la fase di modellazione e mapping. La modellazione consiste nel mappare i dati su un'ontologia in modo che KarmaLinker possa successivamente integrarli con dati provenienti da altre fonti e pubblicarli in un nuovo formato. Per fare questo è necessario associare le colonne del worksheet con dei tipi semantici. Un tipo semantico definisce una relazione tra una colonna presente nei dati ed una proprietà di una classe dell'ontologia.

La modellazione dei dati verrà visualizzata come un albero in cui le foglie sono le colonne, i nodi interni sono le classi dell'ontologia e gli archi sono le proprietà associate alle classi.

Per assegnare un tipo semantico ad una colonna, l'utente deve cliccare sul nome della colonna e selezionare l'opzione *Set Semantic Type*. Successivamente verrà visualizzato un menù che permette all'utente di:

- Associare alla colonna selezionata una proprietà di una classe dell'ontologia
- Rendere la colonna l'URI di una classe dell'ontologia

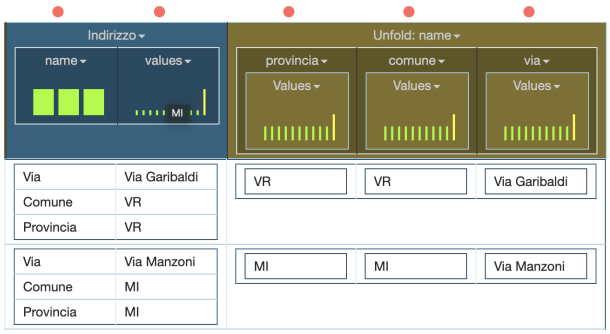


Figura 6.19: Esempio di risultato di Unfold.

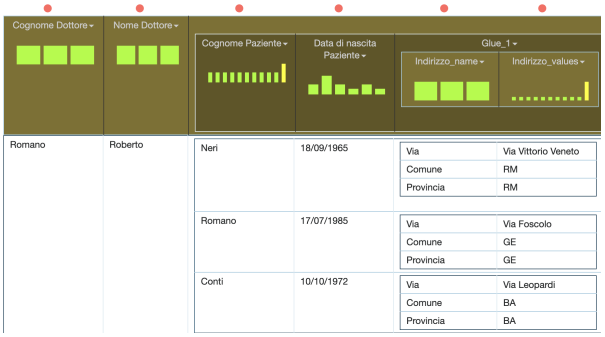


Figura 6.20: Esempio di risultato di Group By.

- Scegliere il literal type, ovvero il tipo di dato
- Scegliere la lingua in cui il valore è fornito

Associare ad ogni classe un URI è fondamentale per una corretta integrazione dei dati. Un URI (Uniform Resource Identifier) è un identificatore univoco essenziale per riferirsi con precisione a specifiche entità all'interno di un dataset. Esso è fondamentale per assegnare a ogni entità un identificatore unico, soprattutto quando si gestiscono più dataset che possono contenere riferimenti alla stessa entità. Nel contesto RDF (Resource Description Framework), gli URI garantiscono che tutte le informazioni su una determinata entità siano collegate allo stesso identificatore, facilitando l'integrazione e l'incrocio di dati tra dataset diversi. Questo contribuisce alla consistenza e all'interoperabilità dei dati tra sistemi differenti. È possibile specificare gli URI per le entità in un dataset o, se non specificati, il sistema genererà nodi vuoti, che sono utili per raggruppare informazioni non corrispondenti a singole entità reali.

La schermata di assegnazione del tipo semantico ad una colonna è illustrata in Figura 6.21.

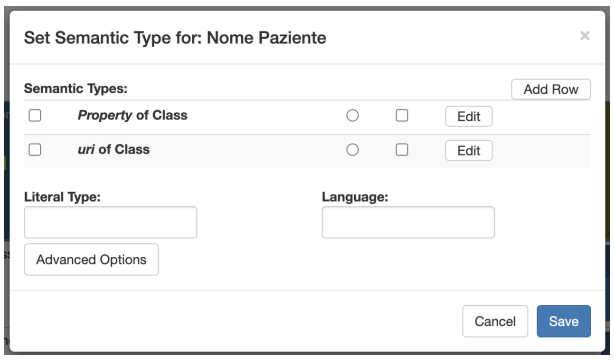


Figura 6.21: Schermata di Set Semantic Type.

Dopo aver scelto se la colonna corrisponde a una proprietà di una classe o all'URI di una classe, viene visualizzata una schermata che permette all'utente di navigare tra tutte le classi e le proprietà

presenti nell'ontologia, in cui effettuare la scelta del tipo semantico.

Questa schermata, prima del mio tirocinio, si limitava a mostrare un elenco dei nomi delle classi e delle proprietà. Questo rendeva l'interfaccia poco usabile poiché il nome della classe non descrive adeguatamente la sua natura. Grazie all'interfacciamento con il servizio SCROLL, e quindi con UKC, ho implementato la visualizzazione di un pop-up quando l'utente posiziona il mouse su un concetto o una proprietà durante la selezione del tipo semantico. Questo pop-up contiene la descrizione del concetto ottenuta tramite una chiamata API al microservizio SCROLL. In questo modo, l'utente risulta notevolmente assistito durante la selezione dei tipi semantici.

Nella prima implementazione che ho realizzato, le descrizioni erano disponibili di default solamente in lingua inglese. Successivamente, dopo aver aggiunto l'opzione per la scelta della lingua, come sarà spiegato in seguito, è stato possibile visualizzare le descrizioni dei concetti nella lingua preferita dall'utente, migliorando ulteriormente l'usabilità complessiva.

Un esempio di pop-up contenente la descrizione di un concetto è in Figura 6.22.

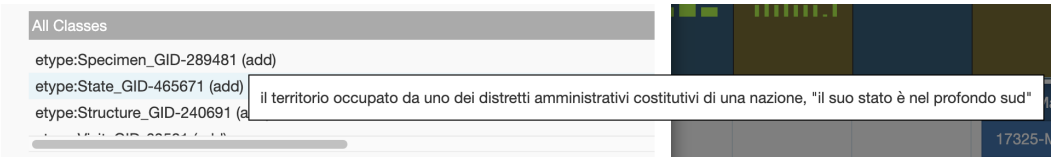


Figura 6.22: Esempio di pop-up.

Quando l'utente seleziona un tipo semantico per una colonna, il mapping viene visualizzato attraverso una struttura a grafo. Un esempio del grafo che viene costruito è illustrato nella Figura 6.23, dove alla colonna denominata "Nome Paziente" viene associata una proprietà che corrisponde al nome

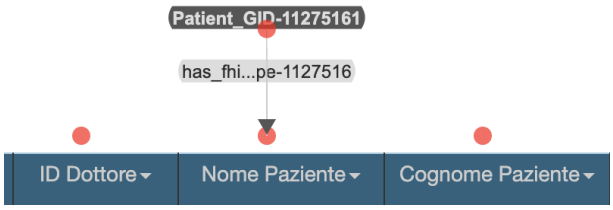


Figura 6.23: Esempio di mapping.

della classe *Patient*.

Nel caso in cui, successivamente, venga assegnata ad un'altra colonna una proprietà che si riferisce a un'entità già menzionata nel mapping, il grafo semantico unirà le due proprietà sullo stesso nodo della classe. Un esempio è mostrato nella Figura 6.24, dove viene scelta un'altra colonna come proprietà di

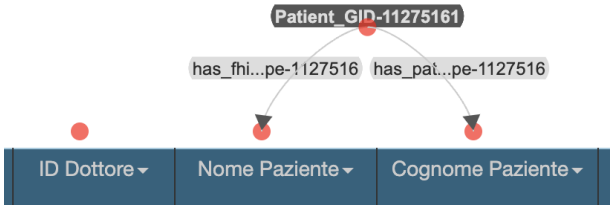


Figura 6.24: Esempio di mapping di due proprietà della stessa entità.

cognome rispetto alla entità *Patient* già esistente.

Nel caso in cui si debbano assegnare proprietà che appartengono alla stessa classe ma a diverse entità, poiché il dataset descrive entità differenti appartenenti alla stessa classe nella stessa riga, il grafo semantico manterrà separati i nodi per indicare chiaramente che si sta facendo riferimento a entità distinte. Un esempio è illustrato nella Figura 6.25, dove le entità associate alle proprietà del dottore e del paziente, pur appartenendo entrambe a *Person*, sono collegate a nodi separati per indicare che si trattano di due entità distinte. Pertanto, l'utente deve specificare correttamente se associare una

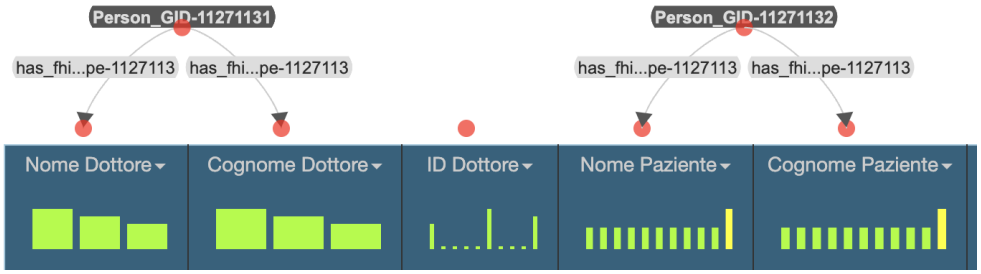


Figura 6.25: Esempio di mapping di due entità della stessa classe.

proprietà ad una entità già esistente della classe oppure creare un nuovo nodo, in base al significato delle colonne.

6.3 Applicazione di un modello

Una volta completata l'attività di modellazione e dopo aver eseguito tutte le operazioni necessarie di trasformazione e modellazione sul foglio di lavoro, è possibile esportare il modello per riutilizzarlo successivamente e applicare le stesse operazioni di mapping ad altri dataset. Per esportare il modello, è necessario innanzitutto selezionare tutte le operazioni che si desidera esportare dalla finestra dei comandi, come mostrato in Figura 6.26. Successivamente, l'utente deve cliccare sul menù *Commands*



Figura 6.26: Selezione delle operazioni da esportare.

e selezionare l'opzione *Export* per procedere con l'esportazione del modello. Come per ogni file generato da KarmaLinker, accanto al nome del worksheet apparirà il file appena esportato. Sarà poi possibile cliccare sul file per aprire il modello in un'altra finestra del browser, oppure fare clic destro per scaricarlo localmente, come mostrato in Figura 6.27.

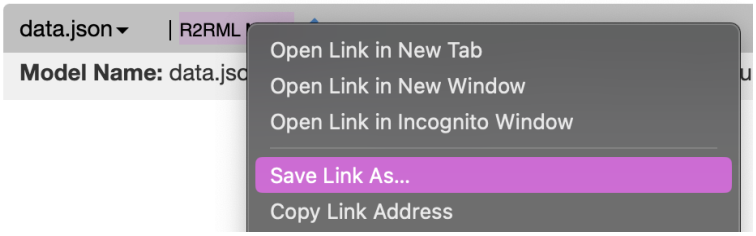


Figura 6.27: Download del modello.

Questo modello potrà essere utilizzato in futuro per altri dataset in modo semplice, evitando di dover eseguire manualmente tutte le operazioni di integrazione necessarie.

Per applicare un modello già esistente a un nuovo dataset, è necessario importare i nuovi dati in KarmaLinker creando un nuovo worksheet, che sarà posizionato sotto quello precedente. Successivamente, cliccando sul nome del nuovo worksheet e selezionando l'opzione *Apply R2RML Model*, sarà possibile indicare a KarmaLinker il modello da applicare, importandolo da file o specificando direttamente l'URL del modello. In Figura 6.28 è illustrato il risultato dell'applicazione del modello precedente a un altro set di dati. Successivamente sarà possibile eseguire tutte le ulteriori operazioni

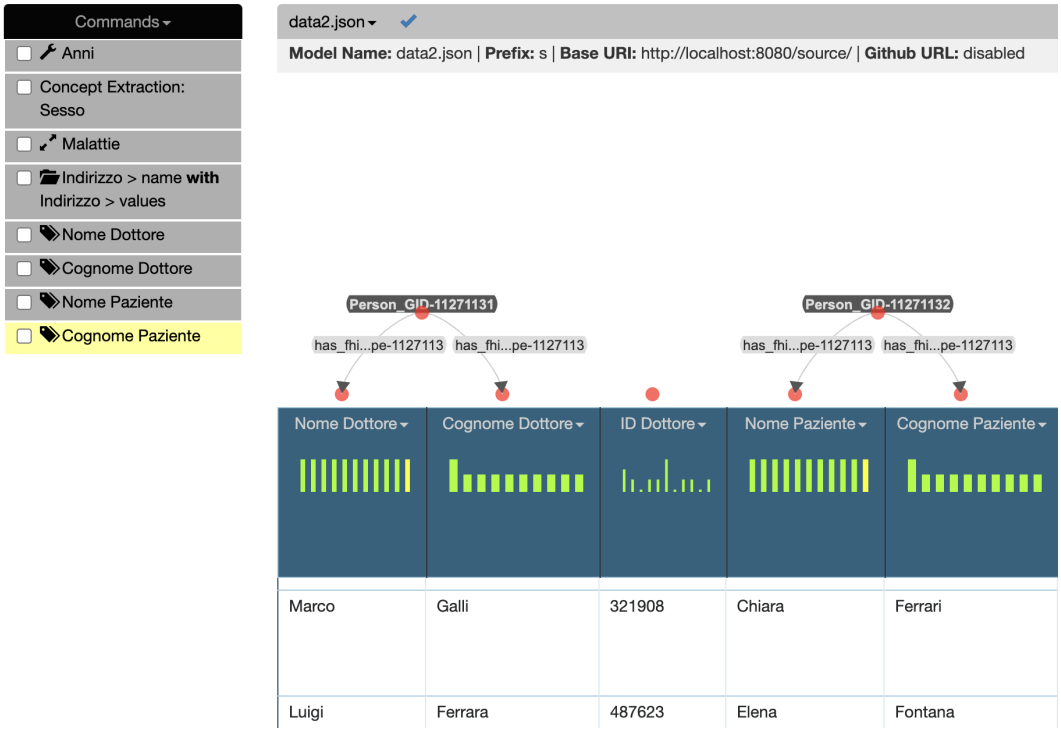


Figura 6.28: Applicazione del modello.

necessarie prima di procedere con l'esportazione dei dati.

Alla fine, per eseguire l'esportazione dei dati e utilizzarli per il loro scopo, è necessario cliccare sul nome del worksheet e selezionare l'opzione *Publish*. Successivamente sarà possibile scegliere il formato desiderato per l'esportazione dei dati. Come avviene con l'esportazione del modello, accanto al nome del worksheet comparirà il file esportato e l'utente potrà aprire il file dei dati integrati in un'altra finestra oppure scaricarlo direttamente facendo clic destro.

6.4 Impostazioni del programma

KarmaLinker utilizza diversi file per eseguire le operazioni richieste e memorizzare informazioni utili. Tutti questi file sono suddivisi in cartelle, che sono raggruppate all'interno di un'unica cartella `{user.home}/karma`. Questa cartella include le seguenti sottodirectory:

- `config`: Questa cartella contiene i file aventi le configurazioni di KarmaLinker. Per attivare o disattivare delle funzionalità bisogna andare a modificare i file presenti in questa cartella.
- `models-autosave`: Questa cartella contiene i modelli, che vengono creati durante l'utilizzo di KarmaLinker, salvati in maniera automatica. Questo è utile nel caso in cui il programma interrompa l'esecuzione per qualsiasi motivo, rendendo possibile il ripristino del lavoro nel punto in cui è stato interrotto. I modelli dunque vengono salvati in questa cartella ogni volta che viene eseguito un comando, con un nome che corrisponde al nome del file su cui si sta lavorando.
- `preloaded-ontologies`: Questa cartella contiene le ontologie che l'utente vuole siano caricate all'interno dello workspace ogni volta che viene avviato KarmaLinker. Dunque sarà compito

dell'utente scegliere quali ontologie caricare in questa cartella per averle a disposizione all'interno dell'ambiente di lavoro di default.

- `python`: Questa cartella contiene i file necessari per la definizione di librerie Python personalizzate, utilizzabili nei modelli KarmaLinker.
- `numeric-semantic-type-files`: Questa cartella contiene i file che si occupano di memorizzare i tipi semantici appresi per i tipi di dati numerici.
- `textual-semantic-type-files`: Questa cartella contiene i file che si occupano di memorizzare i tipi semantici appresi per i tipi di dati testuali.
- `user-preferences`: Questa cartella contiene i file destinati a memorizzare le preferenze utente. Le preferenze dell'utente riguardano la quantità di informazioni che l'utente vuole visualizzare a schermo. Tra queste sono presenti il numero di righe da visualizzare per ogni file, il numero massimo di caratteri per cella, ecc. In questa cartella sono sia memorizzate le preferenze per ogni utente (per ogni workspace) sia le preferenze di default da applicare ad ogni nuovo utente.
- `user-uploaded-files`: Questa cartella contiene la copia di tutti i file che sono stati caricati dall'utente all'interno di KarmaLinker.

All'interno della cartella `{user.home}/karma` sono presenti altre sottocartelle, non menzionate nell'elenco precedente, che però non sono di interesse per l'utente in quanto contengono file utilizzati per scopi interni di KarmaLinker.

KarmaLinker implementa un componente che permette di imparare l'assegnamento dei tipi semantici alle colonne dei dati. Per il funzionamento di questo componente KarmaLinker memorizza le informazioni riguardanti i tipi semantici nelle cartelle `numeric-semantic-type-files` e `textual-semantic-type-files`.

Un'altra cartella di fondamentale importanza è `root/publish`. In questa cartella vengono salvati tutti i file esportati da KarmaLinker. Questa cartella non risiede all'interno di `{user.home}/karma` poiché i file esportati sono resi visualizzabili attraverso dei link nel browser, come visto precedentemente, quindi devono essere accessibili da un percorso web. La cartella contiene diverse sottocartelle, ciascuna dedicata a un tipo di file esportato da KarmaLinker:

- CSV
- JSON
- R2RML
- RDF
- EML

Attraverso l'interfaccia di KarmaLinker è possibile modificare alcune preferenze per l'utilizzo del software. Nella figura Figura 6.29 è illustrato il menù delle impostazioni di KarmaLinker.

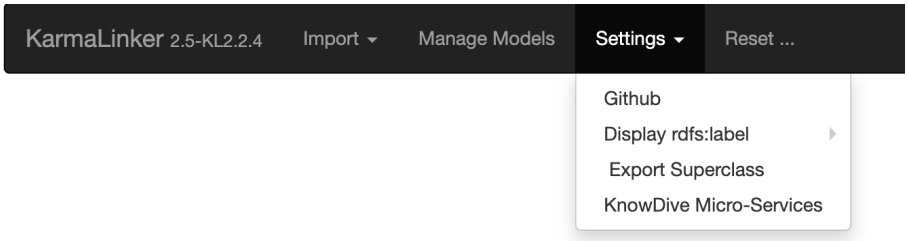


Figura 6.29: Menù delle impostazioni di KarmaLinker.

All'interno di questo menu è possibile scegliere le preferenze di visualizzazione dei dati e decidere se esportare anche le superclassi. È inoltre possibile accedere con un account GitHub per condividere

e versionare i dati e i modelli trattati, oltre che inserire gli URL dei servizi esterni utilizzabili da KarmaLinker.

Tra i servizi esterni di KnowDive, il gruppo di ricerca interno all'Università di Trento presso il quale ho svolto il tirocinio, è possibile specificare l'URL delle SCROLL API, già menzionate e spiegate in precedenza. Durante il mio tirocinio, ho sviluppato una funzionalità che consente di selezionare la lingua desiderata per le risposte della SCROLL API, che influisce sulla lingua dei nomi e delle descrizioni dei concetti restituiti, come è stato spiegato precedentemente. Le lingue disponibili e selezionabili dipendono dalle lingue supportate dalla SCROLL API scelta. La schermata per la selezione dell'URL delle SCROLL API e della lingua è illustrata in Figura 6.30.

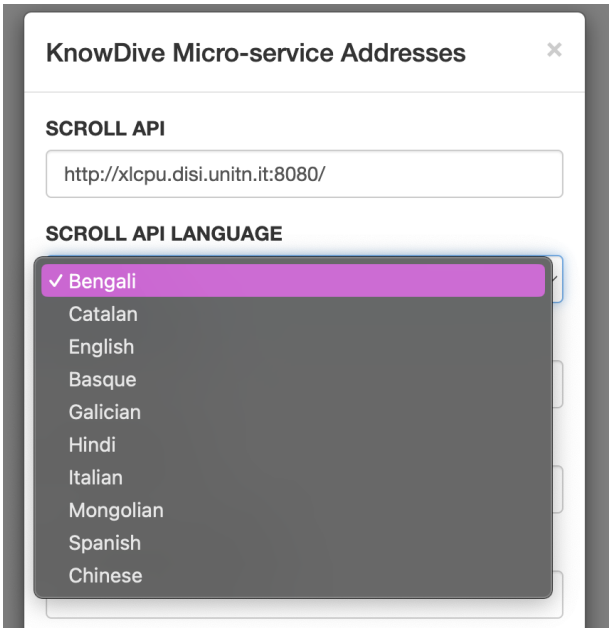


Figura 6.30: Menù dell'inserimento degli URL di servizi esterni.

A fianco del menù delle impostazioni è presente il pulsante di reset. Tramite questa opzione, l'utente può decidere di eliminare alcuni file che KarmaLinker ha salvato in maniera automatica. Più nello specifico, è possibile eliminare tutti i file riguardanti gli assegnamenti dei tipi semantici, tutti i file riguardanti i modelli salvati automaticamente e tutti i file riguardanti gli assegnamenti dei tipi semantici imparati. L'utente può scegliere quali di questi file eliminare, come rappresentato in Figura 6.31.



Figura 6.31: Menù di reset.

6.5 Documentazione API backend

Come ultima parte del mio lavoro di tirocinio, ho documentato tutte le API e gli endpoint forniti dal backend utilizzate per l'interazione tra backend e frontend. Questa documentazione è essenziale per comprendere lo scopo delle chiamate che il frontend effettua al backend e consente all'utente di utilizzare le funzionalità di KarmaLinker senza dover necessariamente usufruire dell'interfaccia fornita dal frontend.

Per questa documentazione, ho utilizzato il linguaggio di specifica OpenAPI, e il risultato è illustrato in Figura 6.32.

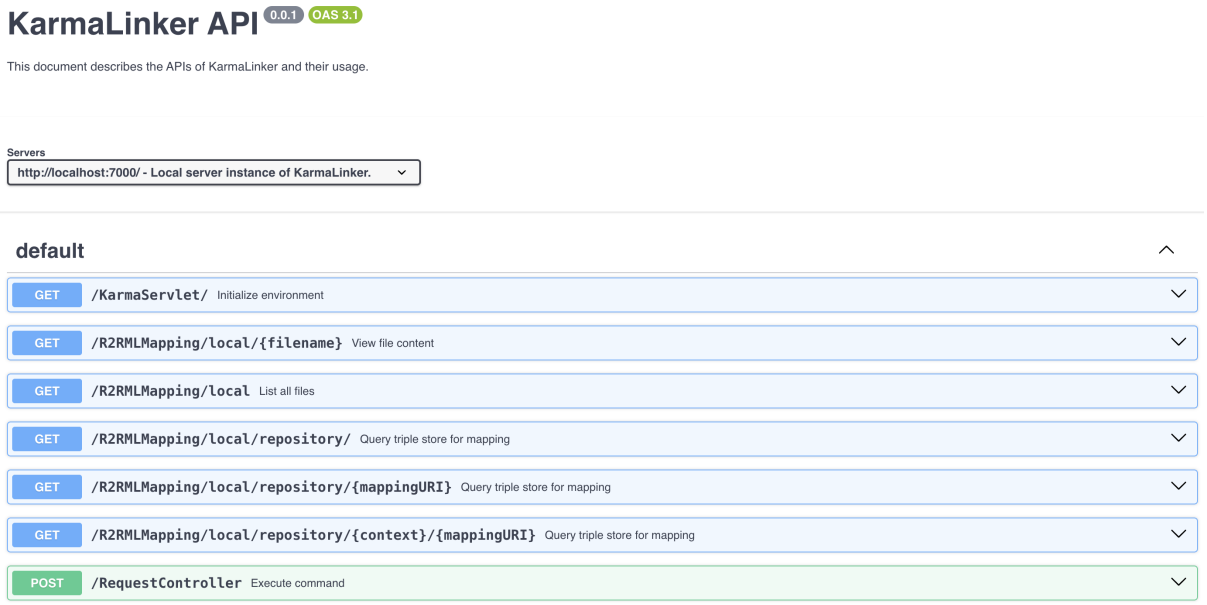


Figura 6.32: Documentazione OpenAPI.

Una breve descrizione dei vari endpoint è la seguente:

- `/KarmaServlet`: Inizializza l'ambiente.
- `/R2RMLMapping/local`: Visualizza il contenuto dei file e elenca tutti i file.
- `/R2RMLMapping/local/repository`: Esegue il comando di interrogazione del triple store per il mapping.
- `/RequestController`: Esegue comandi.
- `/KMLFileTransferHandler`: Carica un file.
- `/ConceptExtractionHandler`: Proxy per il servizio di pipeline di NLP.
- `/HTTPRequestHandler`: Proxy per inoltrare le richieste GET.
- `/services/`: Recupera il modello specificato.
- `/SpatialReferenceSystemService`: Trasforma il sistema di riferimento spaziale di una geometria.
- `/GetConceptDescription`: Ottiene la descrizione di un concetto.
- `/GetOntologyScroll`: Recupera l'ontologia dall'API SCROLL.

Gli ultimi due endpoint, ovvero `/GetConceptDescription` e `/GetOntologyScroll`, sono stati creati e sviluppati da me durante il mio tirocinio per abilitare la comunicazione con il backend ed eseguire le funzionalità che ho aggiunto.

7 Conclusioni

L'integrazione dei dati provenienti da fonti diverse rappresenta una sfida complessa e cruciale nell'era dei Big Data e dell'intelligenza artificiale. La diversità semantica, ossia la rappresentazione differente della stessa entità in diversi database, costituisce l'ostacolo principale a un'integrazione efficace.

Tuttavia, attraverso l'adozione di metodologie e tecnologie adeguate, è possibile affrontare con successo queste difficoltà, garantendo risultati di alta qualità e riducendo i costi associati ai processi di integrazione.

In questo elaborato è stata sviluppata e analizzata una metodologia denominata iTelos, progettata per ottimizzare l'integrazione dei dati attraverso un approccio multilingua e semantico. La struttura di questa metodologia permette di superare le problematiche legate all'eterogeneità semantica, assicurando che i dati integrati siano condivisibili e riutilizzabili per futuri processi di integrazione. Un elemento centrale della metodologia iTelos è l'uso di lessici multilingua (MLDB), che offrono uno spazio semantico imparziale e relazioni interlinguistiche. Questi strumenti consentono di mettere in relazione concetti lessicali tra diverse lingue, eliminando barriere linguistiche e promuovendo l'inclusività digitale.

Uno dei contributi significativi di questa tesi è stata la descrizione e l'utilizzo di KarmaLinker, un software per l'integrazione semantica dei dati. KarmaLinker facilita l'assegnazione della semantica ai dati attraverso una semplice interfaccia grafica, migliorando l'efficienza e la precisione del processo di integrazione. Durante il mio tirocinio, ho contribuito all'espansione delle funzionalità di KarmaLinker e alla documentazione del codice esistente, assicurando che il software possa essere utilizzato efficacemente per futuri progetti.

In sintesi, questa tesi ha dimostrato che l'uso di metodologie semantiche e multilingua, come iTelos e KarmaLinker, può significativamente migliorare l'efficacia e l'efficienza dei processi di integrazione dei dati. Questi strumenti non solo risolvono le problematiche legate all'eterogeneità semantica, ma promuovono anche la condivisibilità e la riutilizzabilità dei dati, aprendo la strada a nuove applicazioni innovative nel campo dell'intelligenza artificiale e oltre. La continua evoluzione di queste tecnologie rappresenta una promessa per il futuro dell'integrazione dei dati e per lo sviluppo di una conoscenza condivisibile e riutilizzabile a livello globale.

Bibliografia

[1] G. Bella, P. Helm, G. Koch, and F. Giunchiglia. Tackling language modelling bias to support linguistic diversity. *acm*, 2024.

[2] 3rdplace. https://3rdplace.com/blog/customer-data/big-data-le-4-v-o-sono-di-piu/. ultimo accesso 30/06/2024.

[3] S. Bocca, M. Dragoni, and F. Giunchiglia. itelos - case studies in building domain specific knowledge graphs. *springer*, 2023.

[4] F. Giunchiglia, K. Batsuren, and A. Alhakim Freihat. One world - seven thousand languages. *springer*, 2017.

[5] F. Giunchiglia, K. Batsuren, and G. Bella. Understanding and exploiting language diversity. *ijcai*, 2017.

[6] F. Giunchiglia, G. Bella, N. Chandran Nair, Y. Chi, and H. Xu. Representing interlingual meaning in lexical databases. *arXiv*, 2023.

[7] F. Giunchiglia, S. Bocca, M. Fumagalli, M. Bagchi, and A. Zamboni. Popularity driven data integration. *arXiv*, 2022.

[8] F. Giunchiglia, A. Zamboni, M. Bagchi, and S. Bocca. Stratified data integration. *arXiv*, 2021.

[9] P. Helm, G. Bella, G. Koch, and F. Giunchiglia. Diversity and language technology: How language modeling bias causes epistemic injustice. *springer*, 2024.

[10] Karma. https://usc-isi-i2.github.io/karma/. ultimo accesso 30/06/2024.

[11] Oracle. https://docs.oracle.com/en/java/javase/22/. ultimo accesso 30/06/2024.

[12] J. Slepicka, C. Yin, P. Szekely, and C. Knoblock. Kr2rml: An alternative interpretation of r2rml for heterogeneous sources. *arXiv*, 2015.

[13] Swagger. https://swagger.io/docs/specification/. ultimo accesso 30/06/2024.

[14] S. Vaskaran. *Java Design Patterns: A Hands-On Experience with Real-World Examples*. Apress, third edition, 2022.

[15] M. Zeni, I. Bison, B. Gauckler, F. Reis, and F. Giunchiglia. Improving time use measurement with personal big data collection – the experience of the european big data hackathon 2019. *arXiv*, 2020.